Relatório Técnico — Conformidade entre documento de requisitos ERL e código gerado por LLM

Disciplina: AED — Processo e Artefato
Grupo: Pedro Blamires Cordeiro, Adriano Faleiro Gregório, Gabriel Borges, Heitor Felipe Trindade Deodato, Jefferson Felex de Faria, Vinícius Rodrigues Gonçalves Ferreira, Gabriel Rodrigues do Vale
Data: 2026-10-15

Resumo

Propomos o processo ERL (Especificação de Requisitos para LLM) e um modelo de documento de requisitos em Markdown com contrato de interface, regras atômicas, ordem de validação de erros e critérios de aceitação concretos. Testamos a hipótese de que código gerado por uma LLM a partir de um documento nesse modelo atinge conformidade de pelo menos 85% com os requisitos. A conformidade foi medida como a fração dos 64 critérios de aceitação aprovados por um oráculo de testes automáticos, validado contra uma implementação de referência e contra 8 mutantes. Em 30 gerações independentes com Claude Opus 5.5, **todas** atingiram 100% de conformidade (64/64). H0 foi rejeitada (Wilcoxon, $p = 2{,}2 \times 10^{-8}$; teste do sinal, $p = 9{,}3 \times 10^{-10}$). A comparação com uma especificação narrativa de mesmo conteúdo também deu 100% em 30 de 30 execuções, de modo que, **neste caso de uso**, não foi possível separar o efeito da estrutura do modelo do efeito do conteúdo: houve efeito teto.

---

1. Introdução e problema

LLMs geram código a partir de texto em linguagem natural, mas não pedem esclarecimentos: diante de uma ambiguidade, escolhem uma interpretação. A qualidade do código gerado depende, então, de quanto a especificação fecha as decisões de antemão. O problema investigado é: **um documento de requisitos estruturado consegue levar uma LLM a produzir software com conformidade de pelo menos 85% em relação aos requisitos?**

2. Hipóteses

Seja C a conformidade de uma execução: a razão entre critérios de aceitação aprovados e o total de critérios do documento (64).

H1 (principal). Código gerado a partir de um documento no modelo ERL tem conformidade acima de 85%.

- $H0_1$: a mediana de C na condição ERL é $\leq 0{,}85$.
- $H1_1$: a mediana de C na condição ERL é $> 0{,}85$.

H2 (secundária). A estrutura do modelo ERL aumenta a conformidade em relação a uma especificação narrativa com o mesmo conteúdo.

- $H0_2$: a conformidade da condição ERL não é estocasticamente maior que a da condição narrativa.
- $H1_2$: a conformidade da condição ERL é estocasticamente maior.

O enunciado da atividade exige H1. Incluímos H2 porque H1 sozinha não separa o mérito do modelo do mérito da LLM.

3. Método

3.1 Desenho

Experimento com um fator (formato da especificação) em dois níveis e execuções independentes.

Variável	Tipo	Definição
Formato da especificação	Independente	**ERL**: `experimento/especificacoes/biblioteca-modelo-erl.md`. **Narrativa**: `experimento/especificacoes/biblioteca-narrativa.md`
Conformidade *C*	Dependente	CAs aprovados ÷ 64
Modelo, prompt, oráculo	Controladas	Iguais nas duas condições (Seção 3.4)

3.2 Objeto do experimento

Um sistema de empréstimos de biblioteca (módulo Python `biblioteca.py`) com 11 requisitos funcionais, 11 regras de negócio, 5 requisitos não funcionais e 64 critérios de aceitação. O domínio foi escolhido por ter regras com interação não trivial (bloqueio por multa ou atraso, fila de reservas, renovações limitadas, teto de multa), fronteiras numéricas e prioridade entre erros. O escopo cabe numa resposta de LLM e é testável sem infraestrutura (ADR 0002).

Condição narrativa (controle). Contém as mesmas regras, as mesmas assinaturas, a mesma ordem de validação e os mesmos valores do documento ERL, escritos em prosa corrida. Não tem IDs, tabelas, glossário, escopo negativo, cenário integrado nem critérios de aceitação. Tudo o que os testes cobram está escrito nela: a diferença entre as condições isola o efeito da **estrutura e dos exemplos**, não o da quantidade de informação.

3.3 Instrumento de medição (oráculo)

`experimento/testes/test_aceitacao.py` contém exatamente um teste por critério de aceitação, nomeado pelo id do CA. O script confere a correspondência um a um antes de cada medição. O módulo gerado é importado como objeto (`m.Nome`), de modo que a falta de um nome reprova só os testes que o usam. Cada teste tem limite de 20 segundos.

Validação do oráculo (`python executar.py validar`), executada antes da coleta:

Verificação	Esperado	Obtido
Implementação de referência	64/64	64/64
Arquivo vazio	0/64	0/64
Mutante: prazo do aluno 15 dias	< 64	50/64
Mutante: sem teto de multa	< 64	63/64
Mutante: limite verificado antes do bloqueio	< 64	63/64
Mutante: atraso inclui a data prevista	< 64	62/64
Mutante: renovação a partir da data da chamada	< 64	61/64
Mutante: fila de reservas ignorada	< 64	62/64
Mutante: consulta devolve o estado interno	< 64	63/64
Mutante: ISBN aceita 12 dígitos	< 64	63/64

O oráculo aprova a implementação correta, não dá crédito a uma implementação vazia e detecta os 8 defeitos injetados. Isso importa para ler os resultados: os 100% observados **não** vêm de testes frouxos, porque um único desvio de regra derruba de 1 a 14 CAs.

3.4 Procedimento

O plano inicial era chamar a API da Anthropic por script (`executar.py gerar`). Sem orçamento para créditos de API, a geração foi feita com **agentes do Claude Code** (modelo Claude Opus 5.5), cada um iniciado sem histórico, com esta instrução fixa:

1. ler **somente** o arquivo da especificação da sua condição, sem abrir nenhum outro arquivo nem executar código;
2. cumprir o prompt do experimento (`experimento/prompt.txt`): *"Implemente o software descrito no documento de requisitos a seguir. Responda apenas com um único bloco de código Python contendo o arquivo biblioteca.py completo."*;
3. gravar o arquivo em `resultados/<condição>/<execução>/biblioteca.py`, numa única gravação, sem testar.

Depois, `executar.py registrar` rodou o oráculo sobre cada arquivo, sem nenhuma edição manual, e gravou `resultados/execucoes.csv`. A ordem de disparo das 60 execuções seguiu um sorteio com semente fixa (2026), intercalando as condições, em lotes de 12 agentes simultâneos. Houve dois desvios na ordem, por erro na montagem dos lotes: as execuções erl 24 e narrativa 07 foram disparadas um lote depois do previsto. Isso não afeta a comparação, porque todas as execuções usaram a mesma configuração e foram geradas no mesmo intervalo de cerca de 5 minutos (14:45 a 14:50).

Regra de inclusão, definida antes da coleta: toda execução entra na análise. Uma execução sem arquivo, ou com arquivo que não importa, conta pelos CAs que passarem, inclusive zero. Nenhuma execução foi descartada nem repetida.

3.5 Tamanho da amostra e análise estatística

São n = 30 execuções por condição. A conformidade é limitada a [0, 1] e se concentra perto de 1, o que dá uma distribuição assimétrica. Por isso os testes principais são não paramétricos:

Hipótese	Teste principal	Complementares	α
H1	Wilcoxon de postos sinalizados para uma amostra, contra 0,85, unilateral	t de Student para uma amostra; teste do sinal (binomial); IC 95% da média por bootstrap	0,05
H2	Mann-Whitney U, unilateral	Correlação rank-biserial	0,05

O teste do sinal foi acrescentado depois do piloto e **antes** da coleta principal: com o piloto inteiro em 100%, o teste t ficaria indefinido (variância zero) e era preciso um teste complementar robusto a empates.

3.6 Piloto

Cinco execuções na condição ERL, antes da coleta principal, para achar defeitos de especificação (fase F8 do processo).

Execução	Conformidade	CAs reprovados	Causa (F8)	Ação
1 a 5	64/64 (100%) em todas	nenhum	—	nenhuma; documento e oráculo congelados

O piloto não revelou nenhum defeito de especificação. As execuções do piloto ficam em `resultados/piloto/` e **não** entram na amostra principal.

---

4. Ambiente

Item	Valor
Modelo	Claude Opus 5.5, via agentes do Claude Code
Parâmetros de amostragem	Padrão do Claude Code; não controláveis por nós
Python	3.14 (execução dos testes)
Bibliotecas	pytest 9.1, pytest-timeout 2.4, scipy 1.18 (`experimento/requirements.txt`)
Data da coleta	2026-10-15
Consumo	~62 mil tokens por execução ERL e ~47 mil por execução narrativa (cerca de 3,6 milhões no total, contando o piloto), dentro da assinatura; custo direto zero

---

5. Resultados

Dados brutos em `experimento/resultados/execucoes.csv`; código gerado em `experimento/resultados/<condição>/<execução>/`; tabelas geradas por `analisar.py` em `experimento/resultados/resumo.md`.

5.1 Estatística descritiva

Condição	n	Média	DP	Mediana	Mín	Máx	IC 95% da média	Execuções ≥ 85%
ERL	30	100,0%	0,0%	100,0%	100,0%	100,0%	[100,0%; 100,0%]	30/30
Narrativa	30	100,0%	0,0%	100,0%	100,0%	100,0%	[100,0%; 100,0%]	30/30

Nenhuma execução reprovou em nenhum critério de aceitação. Todos os 60 arquivos foram gerados e importaram sem erro.

5.2 Teste de H1

Teste	Estatística	p-valor	Decisão
Wilcoxon de postos sinalizados (principal)	$W = 465{,}0$	$2{,}16 \times 10^{-8}$	rejeita $H0_1$
t de Student para uma amostra	indefinido (variância zero)	—	—
Teste do sinal (binomial)	30 de 30 acima de 0,85	$9{,}31 \times 10^{-10}$	rejeita $H0_1$

H1 é aceita: a conformidade do código gerado a partir do documento ERL é maior que 85%, com $p < 0{,}001$. Na prática, nenhuma das 30 execuções ficou abaixo da meta; o pior caso também foi 100%.

5.3 Teste de H2

Teste	Estatística	p-valor	Tamanho de efeito	Decisão
Mann-Whitney U	indefinido (todas as execuções com o mesmo valor)	—	$r = 0$	não rejeita $H0_2$

H2 não pôde ser sustentada: as duas condições atingiram o teto da métrica.

5.4 Critérios de aceitação mais reprovados

Nenhum. Não houve reprovação em nenhuma das 60 execuções nem nas 5 do piloto.

5.5 Verificação de contaminação

Como os agentes rodaram na mesma máquina que a implementação de referência, conferimos se algum código a copiou. A semelhança textual com a referência (`difflib.SequenceMatcher`, por linhas) foi, em média, de 0,35 na condição ERL e de 0,36 na narrativa, com máximo de 0,55. Os códigos têm estruturas diferentes entre si (dicionários, *dataclasses*, classes internas) e de 257 a 332 linhas. A condição narrativa, cujos agentes não tinham nenhum motivo para procurar a referência, tem a mesma semelhança que a ERL. Isso indica que a semelhança vem do estilo comum do modelo e do vocabulário do documento (`_bloqueado`, `saldo_devedor`), não de cópia.

---

6. Discussão

H1. O resultado atende, com folga, ao critério da atividade: 30 de 30 gerações com 100% de conformidade. O oráculo validado por mutação garante que esse 100% não é um artefato de testes fracos.

H2 e o efeito teto. A especificação narrativa também deu 100%. Há duas leituras possíveis, e os dados não permitem escolher entre elas:

1. **Para um sistema deste tamanho, o conteúdo importa mais que a forma.** A narrativa foi escrita *depois* do documento ERL, pelo mesmo grupo, e por isso já tinha todas as decisões que o processo ERL força a tomar: a ordem dos erros, as fronteiras ("na própria data prevista ainda não há atraso"), as chaves exatas do dicionário de retorno, a regra de que ids falhos não são consumidos. Com esse conteúdo, um modelo forte acerta em qualquer formato.
2. **O caso de uso é fácil demais para discriminar.** Com 11 operações e cerca de 300 linhas de código, o sistema cabe com folga na capacidade do modelo. A diferença entre os formatos pode aparecer em sistemas maiores, com mais interação entre regras ou com modelos menores.

A primeira leitura tem uma consequência importante para o trabalho: **o valor do modelo ERL está no processo de escrevê-lo**. As decisões que garantiram 100% nas duas condições foram tomadas porque o modelo exige uma tabela de ordem de validação, um glossário com fronteiras, uma interface com chaves exatas e um checklist de revisão. Um enunciado narrativo típico, escrito sem esse processo, não teria essas decisões. Este experimento, porém, **não mede isso**, porque o controle foi deliberadamente completo (ADR 0003). Medir esse efeito exige outro desenho (Seção 8).

Granularidade da métrica. Com 64 CAs, cada CA vale 1,56 ponto percentual, e um único defeito de regra custa de 1 a 14 CAs (Seção 3.3). A meta de 85% tolera até 9 CAs reprovados (64 × 0,15 = 9,6). Um único defeito numa regra de grande alcance, como o prazo do aluno (14 CAs, conformidade de 78%), já derruba a execução abaixo da meta, enquanto defeitos de alcance pequeno custam de 1 a 3 CAs. A margem observada (zero defeitos) é, portanto, grande.

---

7. Ameaças à validade

Tipo	Ameaça	Mitigação ou consequência
Interna	**Mesma família de modelo.** O documento, a referência e os testes foram escritos com apoio do Claude, e o código foi gerado pelo Claude. O documento pode estar escrito num estilo que esse modelo entende especialmente bem.	Declarada. Replicar com outro modelo (GPT, Gemini, Llama) é a extensão mais importante (Seção 8).
Interna	**Isolamento dos agentes não é verificável.** Os agentes rodaram na máquina onde estão a referência e os testes. A instrução proibia abrir outros arquivos, mas os logs dos agentes não ficaram disponíveis para auditoria.	Verificação indireta por semelhança textual (Seção 5.5): sem sinal de cópia, e semelhança igual na condição que não tinha motivo para procurar a referência.
Interna	**Parâmetros não controlados.** Pelo Claude Code não é possível fixar esforço de raciocínio, temperatura ou versão exata do modelo.	Todos os agentes usaram a mesma configuração padrão, ao mesmo tempo, o que mantém a comparação entre condições justa. O script de API (`executar.py gerar`) permite repetir o experimento com parâmetros fixos.
Interna	Execuções em lotes paralelos, não estritamente na ordem sorteada.	As condições foram intercaladas em cada lote; não houve diferença entre condições a ser confundida com o tempo.
Constructo	A conformidade mede só o que os 64 CAs cobrem. Legibilidade e manutenibilidade não são medidas.	Os CAs cobrem todo RF, toda linha de ordem de validação e toda fronteira numérica. A métrica mede **aderência ao documento**, que é o que a atividade pede.
Constructo	O controle narrativo contém todas as decisões do ERL, algo raro num enunciado real.	Escolha deliberada (ADR 0003): torna H2 um teste conservador. Explica o resultado nulo de H2.
Externa	Um único sistema, pequeno, numa única linguagem.	Não generalizar para sistemas grandes, com persistência, interface ou integração.
Conclusão	Variância zero: o intervalo de confiança degenera e o teste t fica indefinido.	Testes não paramétricos (Wilcoxon e sinal) continuam válidos. A conclusão é sobre **este** modelo e **este** sistema.

---

8. Melhorias propostas

No processo e no artefato (identificadas durante a construção):

1. **Oráculo como fase obrigatória.** Sem um teste por CA, validado com mutantes, não há como saber se uma reprovação é da LLM ou do teste. Por isso a fase F5 virou obrigatória.
2. **Situação inicial explícita em cada CA.** Na revisão (F4) do primeiro rascunho, dois CAs estavam ambíguos: o CA-RF05-06 herdava uma "situação inicial" que contradizia o próprio enunciado, e o CA-RNF03-01 falava do "livro `i`" sem dizer qual era o ISBN. Os dois foram reescritos, e a regra "diga o estado inicial completo ou remeta a uma situação comum" entrou nas instruções (Seção 4).
3. **Permissões explícitas.** A RN-08 ("pode ter dois exemplares do mesmo livro") surgiu da leitura adversarial: sem ela, as duas interpretações eram razoáveis.

No experimento (a partir dos resultados):

4. **Controle realista para H2.** Pedir a pessoas de fora do grupo que escrevam a especificação do mesmo sistema "do jeito que fariam", a partir da mesma entrevista com o cliente, e comparar com quem segue o processo ERL. Isso mede o efeito do **processo** (forçar decisões), que é onde a discussão aponta o valor do modelo.
5. **Casos de uso mais difíceis.** Um sistema maior (20 ou mais RFs), com persistência ou API HTTP, para sair do teto.
6. **Outros modelos.** Repetir com modelos de outros fornecedores e com modelos menores, nos quais a diferença entre formatos tende a aparecer antes.
7. **Medir o esforço.** Registrar as horas gastas para preencher o modelo, para comparar custo e ganho de conformidade.

---

9. Conclusão

O processo ERL e o modelo de documento atingiram o objetivo da atividade: em 30 gerações independentes, o código produzido pelo Claude Opus 5.5 a partir do documento teve 100% de conformidade com os 64 critérios de aceitação, acima da meta de 85% (Wilcoxon, $p = 2{,}2 \times 10^{-8}$). O oráculo foi validado por mutação, então esse resultado não decorre de testes fracos.

A comparação com uma especificação narrativa de mesmo conteúdo também deu 100%, e por isso o experimento não mostra que a **forma** do documento, sozinha, aumente a conformidade neste caso. A principal lição é que, para uma LLM forte, a conformidade depende de as decisões estarem **tomadas e escritas**: ordem de erros, fronteiras, nomes e formatos exatos. O modelo ERL é a ferramenta que obriga o analista a tomá-las. Medir esse efeito do processo, e não só da forma, é o próximo passo.

Referências

- ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering.*
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018. Capítulo 4 (Engenharia de requisitos).
- WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin: Springer, 2012.
- NORTH, D. *Introducing BDD*, 2006 (formato Dado/Quando/Então).

Apêndices

- A. Modelo do documento: `entregaveis/2-modelo-documento-requisitos.md`
- B. Documento preenchido (ERL): `experimento/especificacoes/biblioteca-modelo-erl.md`
- C. Especificação narrativa (controle): `experimento/especificacoes/biblioteca-narrativa.md`
- D. Prompt: `experimento/prompt.txt`
- E. Oráculo: `experimento/testes/test_aceitacao.py`

- F. Dados brutos: `experimento/resultados/execucoes.csv`, `experimento/resultados/resumo.md` e o código gerado em `experimento/resultados/<condição>/<execução>/`